To retain the inheritance semantics, the base class attributes are going to be merged with the subclass ones, so the associated subclass entity table contains both. This is similar to the table-per-class inheritance strategy, with the distinction that the base class is not mapped to a database table (hence, it cannot be used in polymorphic queries or associations).

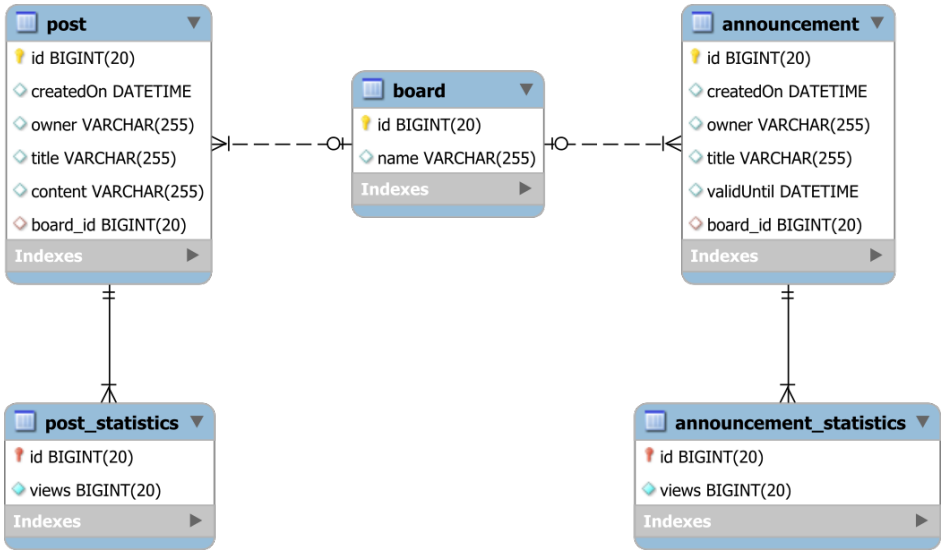

Figure 11.6: Mapped superclass

The `post` and the `announcement` tables feature columns that were inherited from the `Topic` base class. When persisting the same `Post` and `Announcement` entities and saving statistics using the newly defined entity classes:

```
TopicStatistics postStatistics = new PostStatistics(post);
postStatistics.incrementViews();
entityManager.persist(postStatistics);

TopicStatistics announcementStatistics =
    new AnnouncementStatistics(announcement);
announcementStatistics.incrementViews();
entityManager.persist(announcementStatistics);
```

Hibernate generates the following SQL statements:

```
INSERT INTO post (board_id, createdOn, owner, title, content, id )
VALUES (1, '2016-01-17 09:11:07.525', 'John Doe', 'Inheritance',
    'Best practices', 1)
```

```sql
INSERT INTO announcement (board_id, createdOn, owner, title, validUntil, id)
VALUES (1, '2016-01-17 09:11:07.525', 'John Doe', 'Release x.y.z.Final',
    '2016-02-17 09:11:07.529', 2)

INSERT INTO post_statistics (views, id) VALUES (1, 1)

INSERT INTO announcement_statistics (views, id) VALUES (1, 2)
```

Querying for statistics require specifying the actual `Topic` subclass statistics:

```java
PostStatistics statistics = entityManager.createQuery(
    "select s from PostStatistics s join fetch s.topic t where t.id = :postId",
        PostStatistics.class)
.setParameter("postId", postId)
.getSingleResult();
```

This entity query generates a single SQL join statement:

```sql
SELECT
    s.id AS id1_4_0_,
    p.id AS id1_3_1_,
    s.views AS views2_4_0_,
    p.board_id AS board_id6_3_1_,
    p.createdOn AS createdO2_3_1_,
    p.owner AS owner3_3_1_,
    p.title AS title4_3_1_,
    p.content AS content5_3_1_
FROM post_statistics s
INNER JOIN post p ON s.id = p.id
WHERE p.id = 1
```

Polymorphic queries against the `Topic` class are not permitted and the `Board` entity cannot define a `@OneToMany Topic` collection either.

Performance considerations

Although polymorphic queries and associations are no longer permitted, the `@MappedSuperclass` yields very efficient read and write operations. Like single and table-per-class inheritance, write operations require a single insert statement and only one table is used for reading data.

Inheritance best practices

All the inheritance mapping models above require trading something in order to accommodate the impedance mismatch between the relational database system and the object-oriented Domain Model.

The default single table inheritance performs the best in terms of reading and writing data, but it forces the application developer to overcome the column nullability limitation. This strategy is useful when the database can provide support for trigger procedures, and the number of subclasses is relatively small.

The join table is worth considering when the number of subclasses is higher, and the data access layer does not require polymorphic queries. When the number of subclass tables is large, polymorphic queries require many joins, and fetching such a result set has a noticeable impact on application performance. This issue can be mitigated by restricting the result set (e.g. pagination), but that only applies to queries and not to `@OneToMany` or `@ManyToMany` associations. On the other hand, polymorphic `@ManyToOne` and `@OneToOne` associations are fine since, in spite of joining multiple tables, the result set can have at most one record only.

Table-per-class is the least effective when it comes to polymorphic queries or associations. If each subclass is stored in a separate database table, the `@MappedSuperclass` Domain Model inheritance might often be a better alternative.

Although a powerful concept, Domain Model inheritance should be used sparingly and only when the benefits supersede trade-offs.

12. Flushing

As explained in the write-based optimizations section, the Persistence Context acts as a transactional write-behind cache. The Hibernate `Session` is commonly referred to as the first-level cache since every managed entity is stored in a `Map`, and, once an entity is loaded, any successive request serves it from the cache, therefore avoiding a database roundtrip.

However, aside from caching entities, the Persistence Context acts as an entity state transition buffer. Both the `EntityManager` and the Hibernate `Session` expose various entity state management methods:

- The `persist` method takes a transient entity and makes it managed.
- The `merge` method copies the internal state of a detached entity onto a freshly loaded entity instance.
- Hibernate also supports reattaching entity instances (e.g. `update`, `saveOrUpdate` or `lock`) which, unlike `merge`, does not require fetching a new entity reference copy.
- To remove an entity, the `EntityManager` defines the `remove` method, while Hibernate offers a `delete` method.

Like any write-behind cache, the Persistence Context requires flushing in order to synchronize the in-memory persistent state with the underlying database. At flush time, Hibernate can detect if a managed entity has changed since it was loaded and trigger a table row update. This process is called dirty checking, and it greatly simplifies data access layer operations.

So, when using JPA, the application developer can focus on entity state changes, and the Persistence Context takes care of the underlying DML statements. This way, the data access layer logic is expressed using Domain Model state transitions, rather than through insert, update, or delete SQL statements.

This approach is very convenient for several reasons:

- Entity state changes being buffered, the Persistence Context can delay their execution, therefore minimizing the row-level lock interval, associated with every database write operation.
- Being executed at once, the Persistence Context can use JDBC batch updates to avoid executing each statement in a separate database roundtrip.

However, having an intermediate write-behind cache is not without challenges and the Persistence Context can be subject to data inconsistencies. Since efficiency is meaningless if effectiveness is being compromised, this chapter aims to analyze the inner-workings of the flushing mechanism, so the application developer knows how to optimize it without affecting data consistency.

12.1 Flush modes

The Persistence Context can be flushed either manually or automatically.

Both `EntityManager` and the Hibernate native `Session` interface define the `flush()` method for triggering the synchronization between the in-memory Domain Model and the underlying database structures. Even so, without an automatic flushing mechanism, the application developer would have to remember to flush prior to running a query or before a transaction commit.

Triggering a flush before executing a query guarantees that in-memory changes are visible to the currently executing query, therefore preventing *read-your-own-writes* consistency issues.

Flushing the Persistence Context right before a transaction commit ensures that in-memory changes are durable. Without this synchronization, the pending entity state transitions would be lost once the current Persistence Context is closed.

For this purpose, JPA and Hibernate define automatic flush modes which, from a data access operation perspective, are more convenient than the manual flushing alternative.

JPA defines two automatic flush mode types:

- `FlushModeType.AUTO` is the default mode and triggers a flush before every query (JPQL or native SQL query) execution and prior to committing a transaction.
- `FlushModeType.COMMIT` only triggers a flush before a transaction commit.

Hibernate defines four flush modes:

- `FlushMode.AUTO` is the default Hibernate API flushing mechanism, and, while it flushes the Persistence Context on every transaction commit, it does not necessarily trigger a flush before every query execution.
- `FlushMode.ALWAYS` flushes the Persistence Context prior to every query (HQL or native SQL query) and before a transaction commit.
- `FlushMode.COMMIT` triggers a Persistence Context flush only when committing the currently running transaction.
- `FlushMode.MANUAL` disables the automatic flush mode, and the Persistence Context can only be flushed manually.

While the `FlushModeType.COMMIT` and `FlushMode.COMMIT` are equivalent, the JPA `FlushModeType.AUTO` is closer to `FlushMode.ALWAYS` than to the Hibernate `FlushModeType.AUTO` (unlike `FlushMode.ALWAYS`, `FlushModeType.AUTO` does not trigger a flush on every executing query).

FlushMode.AUTO SQL query consistency

The default Hibernate-specific `FlushMode.AUTO` employs a smart flushing mechanism. When executing an HQL query, Hibernate inspects what tables the current query is about to scan, and it triggers a flush only if there is a pending entity state transition matching the query table space. This optimization aims to reduce the number of flush calls and delay the first-level cache synchronization as much as possible.

Unfortunately, this does not work for native SQL queries. Because Hibernate does not have a parser for every database-specific query language, it cannot determine the database tables associated with a given native SQL query. However, instead of flushing before every such query, Hibernate relies on the application developer to instruct what table spaces need to be synchronized.

To guarantee SQL query consistency, the application developer can switch to `FlushMode.ALWAYS` (either at the `Session` level or on a query basis)

```
List<ForumCount> result = session.createSQLQuery(
    "SELECT b.name as forum, COUNT (p) as count " +
    "FROM post p " +
    "JOIN board b on b.id = p.board_id " +
    "GROUP BY forum")
.setFlushMode(FlushMode.ALWAYS)
.setResultTransformer( Transformers.aliasToBean(ForumCount.class))
.list();
```

Another alternative is to explicitly set the table spaces affected by the native query:

```
List<ForumCount> result = session.createSQLQuery(
    "SELECT b.name as forum, COUNT (p) as count " +
    "FROM post p " +
    "JOIN board b on b.id = p.board_id " +
    "GROUP BY forum")
.addSynchronizedEntityClass(Board.class)
.addSynchronizedEntityClass(Post.class)
.setResultTransformer( Transformers.aliasToBean(ForumCount.class))
.list();
```

Only the Hibernate native API (e.g. `Session`) uses the smart flushing mechanism. When using the Java Persistence API (e.g. `EntityManager`), Hibernate flushes before every JPQL or native SQL query.

12.2 Events and the action queue

Internally, each entity state change has an associated event (e.g. `PersistEvent`, `MergeEvent`, `DeleteEvent`, etc) which is handled by an event listener (e.g. `DefaultPersistEventListener`, `DefaultMergeEventListener`, `DefaultDeleteEventListener`).

> Hibernate allows the application developer to substitute the default event listeners with custom implementations.

The Hibernate event listeners translate the entity state transition into an internal `EntityAction` that only gets executed at flush time. For this reason, Hibernate defines the following entity actions:

- When a transient entity becomes persistent, Hibernate generates either an `EntityInsertAction` or an `EntityIdentityInsertAction`, therefore triggering a SQL insert statement at flush time. For the identity generator strategy, Hibernate must immediately execute the insert statement because the entity identifier value must be known upfront.
- During flushing, for every modified entity, an `EntityUpdateAction` is generated which, when executed, triggers a SQL update statement.
- When an entity is marked as removed, Hibernate generates an `EntityDeleteAction`. During flushing, the associations marked with orphan removal can also generate an `OrphanRemovalAction` if a child-side entity is being dereferenced. These two actions trigger a database delete statement.

Because the Persistence Context can manage multiple entities, the pending entity actions are stored in the `ActionQueue` and executed at flush time.

> Entity state transitions can be cascaded from parent entities to children, in which case the original parent entity event is propagated to child entities. For example, when cascading the persist entity state transition, Hibernate behaves as if the application developer has manually called the `persist` method on every child entity.

12.2.1 Flush operation order

Towards the end of the Persistence Context flush, when all `EntityAction(s)` are in place, Hibernate executes them in a very strict order.

1. `OrphanRemovalAction`
2. `EntityInsertAction` and `EntityIdentityInsertAction`
3. `EntityUpdateAction`
4. `CollectionRemoveAction`
5. `CollectionUpdateAction`
6. `CollectionRecreateAction`
7. `EntityDeleteAction`.

The following exercise demonstrates why knowing the flush operation plays a very important role in designing the data access layer actions. Considering the following `Post` entity:

```
@Entity
@Table(name = "post", uniqueConstraints =
    @UniqueConstraint(name = "slug_uq", columnNames = "slug"))
public class Post {

    @Id
    @GeneratedValue
    private Long id;

    private String title;

    private String slug;

    //Getters and setters omitted for brevity
}
```

Assuming the database already contains this `post` record:

```
Post post = new Post();
post.setTitle("High-Performance Java Persistence");
post.setSlug("high-performance-java-persistence");

entityManager.persist(post);
```

When removing a `Post` and persisting a new one with the same `slug`:

```
Post post = entityManager.find(Post.class, postId);
entityManager.remove(post);

Post newPost = new Post();
newPost.setTitle("High-Performance Java Persistence Book");
newPost.setSlug("high-performance-java-persistence");
entityManager.persist(newPost);
```

Hibernate throws a `ConstraintViolationException`:

```
INSERT INTO post (slug, title, id) VALUES (`high-performance-java-persistence`,
    `High-Performance Java Persistence Book`, 2)

SqlExceptionHelper - integrity constraint violation:
unique constraint or index violation; SLUG_UQ table: POST
```

Even if the `remove` method is called before `persist`, the flush operation order executes the insert statement first and a constraint violation is thrown because there are two rows with the same `slug` value.

To override the default flush operation order, the application developer can trigger a manual flush after the `remove` method call:

```
Post post = entityManager.find(Post.class, postId);
entityManager.remove(post);
entityManager.flush();

Post newPost = new Post();
newPost.setTitle("High-Performance Java Persistence Book");
newPost.setSlug("high-Performance-java-persistence");
entityManager.persist(newPost);
```

This time, the statement order matches that of the data access operations:

```
DELETE FROM post WHERE id = 1
INSERT INTO post (slug, title, id) VALUES (`high-Performance-java-persistence`,
    `High-Performance Java Persistence Book`, 2)
```

Hibernate only retains the data access operation order among actions of the same type. Even if the manual flush fixes this test case, in practice, an update is much more efficient than a pair of an insert and a delete statements.

12.3 Dirty Checking

Whenever an entity changes its state from transient to managed, Hibernate issues a SQL insert statement. When the entity is marked as removed, a SQL delete statement is issued.

Unlike insert and delete, the update statement does not have an associated entity state transition. When an entity becomes managed, the Persistence Context tracks its internal state and, during flush time, a modified entity is translated to an update statement.

12.3.1 The default dirty checking mechanism

By default, when an entity is loaded, Hibernate saves a snapshot of persisted data in the currently running Persistence Context. The persisted data snapshot is represented by an `Object` array that is very close to the underlying table row values. At flush time, every entity attribute is matched against its loading time value:

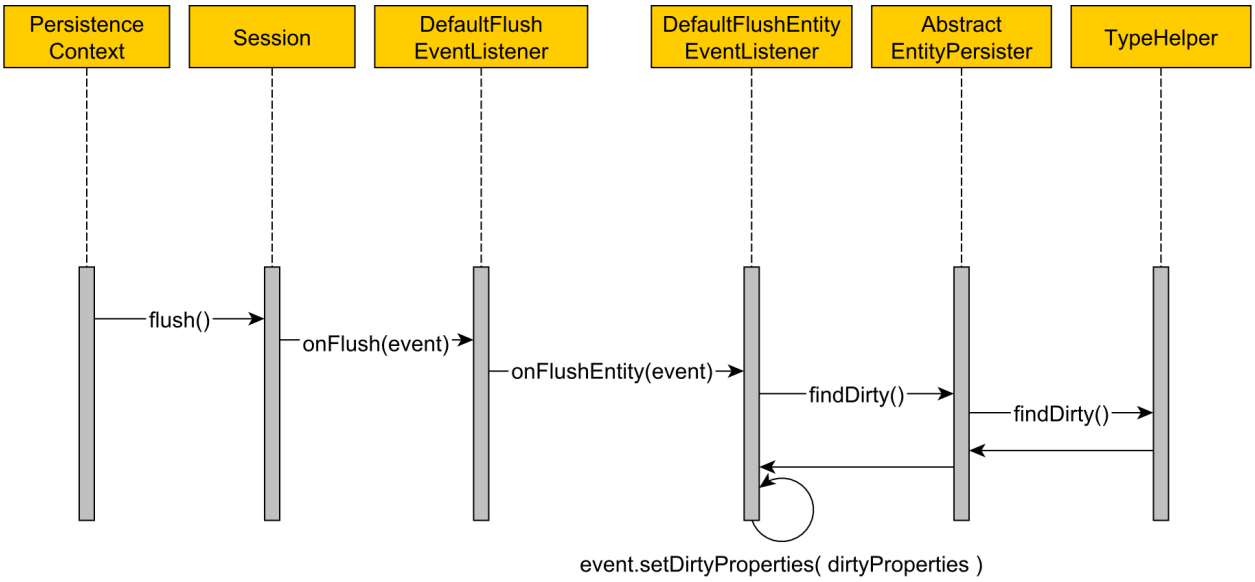

Figure 12.1: Default dirty checking mechanism

The number of individual *dirty checks* is given by the following formula:

$$N = \sum_{k=1}^{n} p_k$$

- n - the number of managed entities
- p - the number of entity attributes.

Even if only one entity attribute changed, Hibernate would still have to go through all managed entities in the current Persistence Context. If the number of managed entities is fairly large, the default dirty checking mechanism might have a significant impact on CPU resources.

12.3.1.1 Controlling the Persistence Context size

Since the entity loading time snapshot is held separately, the Persistence Context requires twice as much memory to store a managed entity.

If the application developer does not need to update the selected entities, a read-only transaction will be much more suitable. From a Persistence Context perspective, a read-only transaction should use a read-only `Session`. By default, the `Session` loads entities in read-write mode, but this strategy can be customized either at the `Session` level or on a query basis:

```
//session-level configuration
Session session = entityManager.unwrap(Session.class);
session.setDefaultReadOnly(true);

//query-level configuration
List<Post> posts = entityManager.createQuery(
    "select p from Post p", Post.class)
.setHint(QueryHints.HINT_READONLY, true)
.getResultList();
```

When entities are loaded in read-only mode, there is no loading time snapshot being taken and the dirty checking mechanism is disabled for these entities.

This optimization addresses both memory and CPU resources. Since the persistent data snapshot is not stored anymore, the Persistence Context consumes half the memory required by a default read-write `Session`. Having fewer objects to manage, the Garbage Collector requires fewer CPU resources when it comes to reclaiming the memory of a closed Persistence Context. Flushing the Persistence Context is also faster and requires fewer CPU resources since the read-only entities are no longer dirty-checked.

When doing batch processing, it is very important to keep the Persistence Context size within bounds. One approach is to periodically *flush* and *clear* the Persistence Context. To avoid the issues associated with a long-running database transaction (e.g. locks being held for long periods of times, database memory consumption), the Java Persistence API allows a Persistence Context to span over multiple database transactions. This way, each batch job iteration clears the Persistence Context, commits the underlying transaction, and starts a new one for the next iteration.

```
EntityManager entityManager = null;
EntityTransaction transaction = null;

try {
    entityManager = entityManagerFactory().createEntityManager();

    transaction = entityManager.getTransaction();
    transaction.begin();

    for ( int i = 0; i < entityCount; ++i ) {
        if ( i > 0 && i % batchSize == 0 ) {
            entityManager.flush();
            entityManager.clear();

            transaction.commit();
            transaction.begin();
        }

        Post post = new Post( String.format( "Post %d", i + 1 ) );
        entityManager.persist( post );
    }

    transaction.commit();
} catch (RuntimeException e) {
    if ( transaction != null && transaction.isActive()) {
        transaction.rollback();
    }
    throw e;
} finally {
    if (entityManager != null) {
        entityManager.close();
    }
}
```

Another approach is to split the load into multiple smaller batch jobs and possibly process them concurrently. This way, long-running transactions are avoided and the Persistence Context has to manage only a limited number of entities.

The Persistence Context should be kept as small as possible. As a rule of thumb, only the entities that need to be modified should ever become managed. Read-only transactions should either use DTO projections or fetch entities in read-only mode.

12.3.2 Bytecode enhancement

Although Hibernate has supported bytecode enhancement for a long time, prior to Hibernate 5, the dirty checking mechanism was not taking advantage of this feature. Hibernate 5 has re-implemented the bytecode instrumentation mechanism, and now it is possible to avoid the reflection-based dirty checking mechanism. The bytecode enhancement can be done at compile-time, runtime or during deployment. The compile-time alternative is preferred for the following reasons:

- The enhanced classes can be covered by unit tests.
- The Java EE application server or the stand-alone container (e.g. Spring) can bootstrap faster because there's no need to instrument classes at runtime or deploy-time.
- Class loading issues are avoided since the application server does not have to take care of two versions of the same class (the original and the enhanced one).

The Hibernate tooling project comes with bytecode enhancement plugins for both Maven and Gradle. For Maven, the following plugin must be configured in the `pom.xml` file:

```
<plugin>
    <groupId>org.hibernate.orm.tooling</groupId>
    <artifactId>hibernate-enhance-maven-plugin</artifactId>
    <version>${hibernate.version}</version>
    <executions>
        <execution>
            <configuration>
                <enableDirtyTracking>true</enableDirtyTracking>
            </configuration>
            <goals>
                <goal>enhance</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

The bytecode enhancement plugin supports three instrumentation options which must be explicitly enabled during configuration:

- lazy initialization (allows entity attributes to be fetched lazily)
- dirty tracking (the entity tracks its own attribute changes)
- association management (allows automatic sides synchronization for bidirectional associations).

After the Java classes are compiled, the plugin goes through all entity classes and modifies their bytecode according to the instrumentation options chosen during configuration.

When enabling the *dirty tracking* option, Hibernate tracks attribute changes through the `$$_hibernate_tracker` attribute. Every setter method also calls the `$$_hibernate_trackChange` method to register the change.

```
@Transient
private transient DirtyTracker $$_hibernate_tracker;

public void $$_hibernate_trackChange(String paramString) {
    if (this.$$_hibernate_tracker == null) {
        this.$$_hibernate_tracker = new SimpleFieldTracker();
    }
    this.$$_hibernate_tracker.add(paramString);
}
```

Considering the following original Java entity class setter method:

```
public void setTitle(String title) {
    this.title = title;
}
```

Hibernate transforms it to the following bytecode representation:

```
public void setTitle(String title) {
    if(!EqualsHelper.areEqual(this.title, title)) {
        this.$$_hibernate_trackChange("title");
    }
    this.title = title;
}
```

When the application developer calls the `setTitle` method with an argument that differs from the currently stored `title`, the change is going to be recorded in the `$$_hibernate_tracker` class attribute.

During flushing, Hibernate inspects the `$$_hibernate_hasDirtyAttributes` method to validate if an entity was modified. The `$$_hibernate_getDirtyAttributes` method returns the names of all changed attributes.

```
public boolean $$_hibernate_hasDirtyAttributes() {
    return $$_hibernate_tracker != null && !$$_hibernate_tracker.isEmpty();
}

public String[] $$_hibernate_getDirtyAttributes() {
    if($$_hibernate_tracker == null) {
        $$_hibernate_tracker = new SimpleFieldTracker();
    }
    return $$_hibernate_tracker.get();
}
```

To validate the bytecode enhancement performance gain, the following test measures the dirty tracking time for 10, 20, 50, and 100 `Post` entity hierarchies (each `Post` is associated with one `PostDetails`, two `PostComment` and two `Tag` entities). Each iteration modifies six attributes: the `Post` title, the `PostDetails` creation date and owner, the `PostComment` review and the `Tag` name.

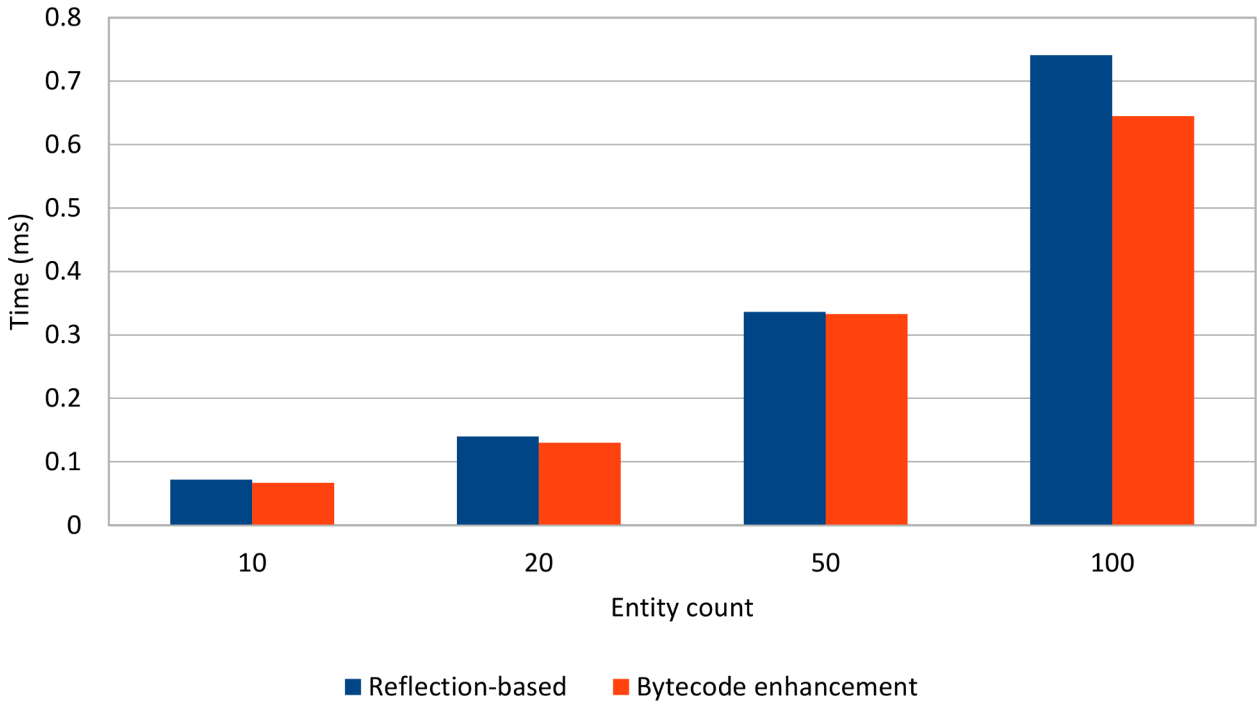

Figure 12.2: Bytecode enhancement performance gain

Both dirty checking mechanisms are very fast, and, compared to how much it takes to run a database query, the in-memory attribute modification tracking is insignificant. Up to 50 `Post` entities, the reflection-based and the bytecode enhancement dirty checking mechanisms perform comparably.

Although bytecode enhancement dirty tracking can speed up the Persistence Context flushing mechanism, if the size of the Persistence Context is rather small, the improvement will not be that significant.

The entity snapshot is still saved in the Persistence Context even when using bytecode enhancement because the persisted data might be used for the second-level cache entries. For this reason, keeping the Persistence Context in reasonable boundaries stays true no matter the dirty tracking mechanism in use.

13. Batching

As explained in the JDBC Batch Updates chapter, grouping multiple statements can reduce the number of database roundtrips, therefore, lowering transaction response time.

When it comes to translating entity state transitions, Hibernate uses only `PreparedStatement(s)` for the automatically generated insert, update, and delete DML operations. This way, the application is protected against SQL injection attacks, and the data access layer can better take advantage of JDBC batching and statement caching.

With plain JDBC, batch updates require programmatic configuration because, instead of calling `executeUpdate`, the application developer must use the `addBatch` and `executeBatch` methods. Unfortunately, performance tuning is sometimes done only after the application is deployed into production, and switching to batching JDBC statements requires significant code changes.

By default, Hibernate doesn't use JDBC batch updates, so when inserting 3 `Post` entities:

```
for (int i = 0; i < 3; i++) {
    entityManager.persist(new Post(String.format("Post no. %d", i + 1)));
}
```

Hibernate executes 3 insert statements, each one in a separate database roundtrip:

```
INSERT INTO post (title, id) VALUES (Post no. 1, 1)
INSERT INTO post (title, id) VALUES (Post no. 2, 2)
INSERT INTO post (title, id) VALUES (Post no. 3, 3)
```

Unlike JDBC, Hibernate can switch to batched `PreparedStatement(s)` with just one configuration property, and no code change is required:

```
<property name="hibernate.jdbc.batch_size" value="5"/>
```

The `hibernate.jdbc.batch_size` configuration is applied globally for all `Session(s)`.

Session-level JDBC batching

Hibernate 5.2 adds support for Session-level JDBC batching. Prior to this release, there was no way to customize the JDBC batch size on a per-business use case basis. However, this feature is really useful since not all business use cases have the same data persistence requirements.

The JDBC batch size can be set programmatically on a Hibernate `Session` as follows:

```
doInJPA(entityManager -> {
    entityManager.unwrap(Session.class).setJdbcBatchSize(10);

    for ( long i = 0; i < entityCount; ++i ) {
        Post post = new Post();
        post.setTitle(String.format("Post nr %d", i));
        entityManager.persist(post);
    }
});
```

If the `EntityManager` uses a `PersistenceContextType.EXTENDED` scope, it is good practice to reset the custom JDBC batch size before existing the current business method:

```
@PersistenceContext(type = PersistenceContextType.EXTENDED)
private EntityManager entityManager;

@TransactionAttribute(value=REQUIRED)
public void savePosts() {
    entityManager.unwrap(Session.class).setJdbcBatchSize(10);
    try {
        for ( long i = 0; i < entityCount; ++i ) {
            Post post = new Post();
            post.setTitle(String.format("Post nr %d", i));
            entityManager.persist(post);
        }
        entityManager.flush();
    } finally {
        entityManager.unwrap(Session.class).setJdbcBatchSize(null);
    }
}
```

By setting the `Session`-level JDBC batch size to `null`, Hibernate is going to use the `SessionFactory` configuration (e.g. `hibernate.jdbc.batch_size`) the next time the `EXTENDED EntityManager` gets reused.

13.1 Batching insert statements

After setting the batch size property, when rerunning the previous test case, Hibernate generates a single insert statement:

```
Query:  ["INSERT INTO post (title, id) VALUES (?, ?)"],
Params: [('Post no. 1', 1), ('Post no. 2', 2), ('Post no. 3', 3)]
```

Identity columns and JDBC batching

If the `Post` identifier used an identity column, Hibernate would disable batched inserts.

```
INSERT INTO post (id, title) VALUES (default, 'Post no. 1')
INSERT INTO post (id, title) VALUES (default, 'Post no. 2')
INSERT INTO post (id, title) VALUES (default, 'Post no. 3')
```

Once an entity becomes managed, the Persistence Context needs to know the entity identifier to construct the first-level cache entry key, and, for identity columns, the only way to find the primary key value is to execute the insert statement.

This restriction does not apply to update and delete statements which can still benefit from JDBC batching even if the entity uses the identity strategy.

Assuming the `Post` entity has a `@OneToMany` association with the `PostComment` entity, and the persist event is cascaded from the `Post` entity to its `PostComment` children:

```
@OneToMany(cascade = CascadeType.ALL, mappedBy = "post", orphanRemoval = true)
private List<PostComment> comments = new ArrayList<>();
```

When persisting three `Post(s)` along with their `PostComment` child entities:

```
for (int i = 0; i < 3; i++) {
    Post post = new Post(String.format("Post no. %d", i));
    post.addComment(new PostComment("Good"));
    entityManager.persist(post);
}
```

Hibernate executes one insert statement for each persisted entity:

```
INSERT INTO post (title, id) VALUES ('Post no. 0', 1)
INSERT INTO post_comment (post_id, review, id) VALUES (1, 'Good', 2)

INSERT INTO post (title, id) VALUES ('Post no. 1', 3)
INSERT INTO post_comment (post_id, review, id) VALUES (3, 'Good', 4)

INSERT INTO post (title, id) VALUES ('Post no. 2', 5)
INSERT INTO post_comment (post_id, review, id) VALUES (5, 'Good', 6)
```

Even if the JDBC batching is enabled, Hibernate still executes each statement separately. This is because JDBC batching requires executing the same `PreparedStatement`, and, since the parent and the child entity persist operations are interleaved, the batch must be flushed prior to proceeding with an entity of different type.

To fix this, the inserts must be ordered while still maintaining the parent-child referential integrity rules. For this purpose, Hibernate offers the following configuration property:

```
<property name="hibernate.order_inserts" value="true"/>
```

With this setting in place, Hibernate can benefit from JDBC batching once more:

```
INSERT INTO post (title, id)
VALUES (Post no. 0, 1), (Post no. 1, 3), (Post no. 2, 5)

INSERT INTO post_comment (post_id, review, id)
VALUES (1, Good, 2), (3, Good, 4), (5, Good, 6)
```

13.2 Batching update statements

Once the `hibernate.jdbc.batch_size` configuration property is set up, JDBC batching applies to SQL update statements too. Running the following test case:

```
List<Post> posts = entityManager.createQuery(
    "select p from Post p ", Post.class)
.getResultList();

posts.forEach(post -> post.setTitle(post.getTitle().replaceAll("no", "nr")));
```